

“Predicting Residential Electricity Consumption Using CNN-BiLSTM-SA Neural Networks”

*Submitted to the Department of Computer Science and Engineering
in partial fulfillment of the requirements for the award of the degree of*

BACHELOR OF TECHNOLOGY

In

**Computer Science and Engineering
of**

Biju Patnaik University of Technology, Odisha

Submitted by

RABI NARAYAN PADHY (Registration No: 2201204070)

Under the Guidance of

Prof. Ramachandra Padhy

Assistant Professor in
Department of Computer Science and Engineering



**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
ROLAND INSTITUTE OF TECHNOLOGY,
SURYA VIHAR, BERHAMPUR, Dist-Ganjam, (Odisha), Pin-761008
SPRING(January-June) 2026**

In memory of

Late Dr J Surya Rao "Founder Chairman" RIT Our deepest gratitude.

BONAFIDE CERTIFICATE

This is to certify that the project titled "Predicting Residential Electricity Consumption Using CNN-BiLSTM-SA Neural Networks" is a Bonafide record of the work done by

RABI NARAYAN PADHY (Registration No: 2201204070)

in partial fulfillment of the requirements of the award of the degree of bachelor of technology in computer science and engineering of the **ROLAND INSTITUTE OF TECHNOLOGY, GOLONTHARA, OD 761008**, during the year 2025-26



Prof. Ramachandra Padhy
Assistant Professor in Dept. of CSE Guide



Dr. Sunil Kumar Sahu
Professor and Head of the Department

Project Viva-voce held on **13 May 2026**



Prof. Parveen Kumar Patro
Assistant Professor in Dept. of CSE
Internal Examiner



Dr. Pabitra Kumar Tripathy
Professor of CSE and Principal, Kalam
Institute of Technology External
Examiner



ACKNOWLEDGEMENTS

I would like to express my deepest gratitude to the following people for guiding me throughout this course and without whom this project and the results achieved from it would not have been possible.

I sincerely thank **Prof. Rama Chandra Padhy**, Professor, Department of Computer Science and Engineering, for helping and guiding me throughout the course of this project. Without his guidance, I would not have been able to successfully complete this project. His patience and genial attitude have been and always will be a source of inspiration to me.

I am grateful to **Prof. Sunil Kumar Sahu**, Head of the Department, Department of Computer Science and Engineering, for allowing me to avail myself of the facilities provided by the department.

I am thankful to the Principal, **Dr. Gunamani Jena**, for providing the necessary facilities in this institution.

I am also thankful to the Honorable Chairperson, **Mrs. J. Jayalakshmi**, for motivating me and for arranging training and internship opportunities related to the project.

Finally, I am thankful to the faculty and staff members of the Department of Computer Science and Engineering, my parents, and my friends for their constant support and help.

INDEX

S.NO.	CHAPTER PAGE	PAGENO
1.	INTRODUCTION	1
2.	HOW MACHINE LEARNING WORKS	2
2.1.1	Machine Learning Algorithms	2
2.1.2	Types of Machine Learning Methods.....	3
2.1.3	Applications of Machine Learning.....	4
2.1.4	WHAT IS DEEP LEARNING?.....	5
2.1.4	How Deep Learning Works.....	6
3.	LITERATURE SURVEY	7
1.	Traditional Statistical Models.....	7
2.	Machine Learning Approaches.....	7
3.	Deep Learning-Based Approaches	8
4.	Hybrid Approaches.....	8
4.	SYSTEM ANALYSIS	9
4.1	EXISTING SYSTEM	9
4.2	PROPOSED SYSTEM	10
5.	FUNCTIONAL REQUIREMENTS	11
5. 1	SYSTEM FUNCTIONAL REQUIREMENTS	11
6.	NON-FUNCTIONAL REQUIREMENTS	13
6.1	REQUIREMENTS	13
7.	FEASIBILITY STUDY	15
8.	REQUIREMENT SPECIFICATION	16
8.1	Hardware Requirements.....	16
8.2	Software Requirements	16
9	SYSTEM DESIGN	17
9.1	METHODOLOGY	17
	Data Collection and Preprocessing.....	17
	Model Architecture - CNN-BiLSTM-SA	17
	Model Training and Evaluation.....	18
	Results and Analysis	18
	Extension with CNN-BIGRU-SA.....	19

9.1 DATASET	19
9.3 SYSTEM ARCHITECTURE	20
10 UML DIAGRAMS	21
10.1 USE CASE DIAGRAM	22
10.2 CLASS DIAGRAM	23
10.3 SEQUENCE DIAGRAM	23
10.4 COLLABORATION DIAGRAM	24
10.5 ACTIVITY DIAGRAM:	25
11. TECHNOLOGY DESCRIPTIONS	27
11.1 PYTHON FEATURES	27
11.2 PYTHON APPLICATIONS	27
11.4 WHY PYTHON	28
12. SAMPLE CODE	29
13. TESTING	38
13.1 INTRODUCTION	38
13.1.1 System Testing and Implementation	38
13.1.2 Testing Techniques	39
13.1.3 Testing Strategies	39
14. SCREENSHOTS	41
15. CONCLUSION	47
16. GLOSSARY	48
16. BIBLIOGRAPHY	50

LIST OF FIGURES

Figure 1 Architecture	20
Figure 2 Use case Diagram	22
Figure 3 Class Diagram	23
Figure 4 Sequence Diagram	24
Figure 5 Collaboration Diagram	25
Figure 6 Activity Diagram	26

LIST OF SCREENS

Screen 1 Electricity Consumption	41
Screen 2 SVM algorithm	41
Screen 3 Linear Regression algorithm	42
Screen 4 CNN-BiLSTM-SA algorithm	42
Screen 5 CNN-BiGRU-SA algorithm	43
Screen 6 RMSE Performance of all algorithms	43
Screen 7 Home Page	44
Screen 8 Forecasting Page	44
Screen 9 Extension CNN-BiGRU-SA Model Output	45

ABSTRACT

The rapid growth in population and the increasing use of energy-intensive household appliances have significantly increased residential electricity consumption. The extensive use of devices such as air conditioners, refrigerators, washing machines, and smart home systems creates major challenges for modern power grids, including load imbalance, peak demand pressure, voltage instability, and localized power outages. These problems reduce the overall efficiency and reliability of electrical power systems. Therefore, accurate short-term electricity load forecasting is essential for effective energy management, stable power distribution, and improved smart grid performance.

This work proposes a hybrid CNN–BiLSTM–Self-Attention (SA) model for residential electricity consumption prediction. The Convolutional Neural Network (CNN) layer extracts important local temporal features and short-term usage patterns from historical electricity data. The Bidirectional Long Short-Term Memory (BiLSTM) layer captures both past and future temporal dependencies, helping the model learn long-term consumption trends more effectively. In addition, the Self-Attention mechanism improves prediction performance by focusing on the most relevant time steps and significant features in the data.

The proposed model is evaluated using the UCI Machine Learning Repository Individual Household Electric Power Consumption Dataset for both hour-ahead and day-ahead forecasting tasks. The model performance is measured using Mean Absolute Error (MAE), Root Mean Square Error (RMSE), and R^2 Score. Experimental results show that the proposed CNN–BiLSTM–SA model outperforms traditional approaches such as CNN, LSTM, BiLSTM, Support Vector Regression (SVR), and Random Forest by achieving lower prediction errors and higher forecasting accuracy.

Overall, the proposed approach provides an effective and reliable solution for short-term electricity load forecasting. It supports better demand-side management, reduces peak load stress, improves energy utilization, and enhances the stability and efficiency of modern smart grid systems.

1. INTRODUCTION

Electricity consumption has become a critical issue due to rapid industrialization, urbanization, and the widespread use of household appliances. The increasing demand in residential sectors places significant pressure on energy supply systems, creating challenges in maintaining a balanced power grid and preventing localized outages. As a result, accurate prediction of electricity consumption has become essential for ensuring a stable, efficient, and sustainable power supply.

Residential electricity consumption accounts for a substantial share of total energy usage. This demand continues to grow due to rising appliance usage, changing lifestyles, and economic development. Accurate prediction of residential electricity consumption plays a key role in optimizing power generation and distribution, enabling utility companies to allocate resources efficiently and reduce the risk of shortages. It also enhances grid stability by maintaining load balance, thereby preventing blackouts and ensuring uninterrupted power supply.

In addition, precise forecasting helps reduce operational costs by minimizing excess energy production and improving resource utilization. It also supports sustainability by enabling better integration of renewable energy sources into the grid. Furthermore, understanding consumption patterns allows utility providers to improve customer service and help users manage their energy usage more effectively.

However, predicting residential electricity consumption remains a complex task due to several challenges. Consumption patterns are highly dynamic and influenced by multiple factors, including weather conditions, time of day, seasonal variations, and individual user behavior. These patterns are nonlinear and continuously evolving, making accurate modeling difficult. Data-related issues such as noise, missing values, and inconsistencies further impact prediction accuracy. Moreover, high-frequency data collected at short intervals generates large volumes of information, requiring advanced analytical techniques for efficient processing and modeling.

2. HOW MACHINE LEARNING WORKS

To begin with, this and computer science and, more particularly, AI and its subset called machine learning focuses on the collection of data and making use of algorithms to accomplish the objective of replicating the human learning processes more accurately rather than attempting to entirely imitate them. This is critical for data science as it allows systems to be able to uncover complex relationships within data that can make predictions or classifications automatically, and later using these relationships to make decisions within a business or any other establishment. Such insights are essential, especially for data mining initiatives and can have critical impact on major business development indices.

However, as the volume of big data is increasing so does the need for data scientists who understand the business perspective and the problems that needs solving and how data can provide the answers to the problems. This is where machine learning becomes relevant as it provides an opportunity to solve all these problems by automating the process through the use of statistical methods in training algorithms to recognize underlying relationships or patterns with data.

2.1.1 Machine Learning Algorithms

There are three important principles on which machine learning algorithms evolve.

1. A Decision Process.

This is the formulization of the problem which has to be solved by the machine learning. The problem solving begins with the identification of what the problem is and this means stating what are the objectives of the problem

2. An Error Function

The algorithm's predictions are ascertained with the aid of the error function. The error function is an evaluation tool, which verifies the efficiency of the model based on the output in relation to the available known situations and indicates places where improvements can be made in the model.

3. A Model Optimization Process

During the optimization process, the already defined model is employed to enhance its accuracy by modifying parameters, for instance, weights, that determine the alignment of the model with the data. The model conducts its own evaluation, causes some changes which are afterward subjected to further evaluation, and repeats everything until it fully satisfies itself with the accuracy of its prediction, or the process exceeds some limits preset.

2.1.2 Types of Machine Learning Methods

Supervised Machine Learning

Initiating with labelled data leads to supervised learning algorithms, thereby informing the input and the output. In this manner, the model is able to understand the rules and the relationships of the data so as to classify patterns and provide predictions of unseen data. Adding the data would energise the model to make weight adjustments to the parameters so as to better the performance during the training phase. Cross-validation is adopted for balancing cases where overfitting (not too general) and underfitting situations (too general) are avoided.

Unsupervised Machine Learning

On the contrary, unsupervised learning involves working with unlabelled sets of data. Such algorithms examine the input data to discover and cluster the data into groups or structures that were not taught to them. This makes it very useful for exploratory data analysis, for market segmentation, cross-marketing, or for recognizing images and patterns.

Semi-Supervised Learning

To do this, semi-supervised learning bridges supervised and unsupervised learning by employing a few labeled data and lots of unlabeled data. The final understanding of the academic efforts is achieved through feature extraction and image collection from the labeled files, hence the model is able to understand the unpreferred files. This model proves useful in situations where the labeled data is scarce or the processes of labeling data is expensive or infeasible.

2.1.3 Applications of Machine Learning

1. Voice Identification

Automatic Speech Recognition (ASR) also known as voice recognition is that aspect of natural language processing (NLP) wherein spoken words are converted to written texts. This technology is quite widely used on portable devices, for instance, mobile phones, where people are able to do voice searches on devices such as Siri and can also send text messages without using their hands.

2. Customer support

Customer interaction and engagement are being revolutionized by AI powered chatbots. Customers can be assisted on sites and across social media platforms. These AI powered bots can answer frequently asked questions about shipping conditions, give fitting and size suggestions, and recommend goods to buy, all of which help to maximize customer interaction. These can include ecommerce virtual agents, slack and facebook messenger bots, and voice interface systems.

3. Vision AI

With the help of computer vision, systems can analyze faces, videos and other visual aspects to gain an understanding of the situation at hand. Unlike simple image recognition, such approaches can view pictures and videos and consider them for decision making. These include:

- Tagging of pictures posted on social media sites.
- Healthcare radiology imaging.
- Automobiles with automated driving features in the automobile sector.

4. Virtual shopping assistants

Past behavioral data of users is outputted by an AI algorithm and then analyzed to find valuable insights which can be used to strategically target users. This is most common in e-commerce stores where clients are offered recommendations about items that can be purchased in addition to what the user initially intended to buy.

5. Stock trading robots

The new trading platforms that have been developed focus on algorithmic trading of stocks while an AI powered engine assembles the stock portfolio. Daily, these computers can place hundreds of thousands or millions of trades, making them suitable for stock markets.

2.1.4 WHAT IS DEEP LEARNING?

Deep learning is a subfield of AI that engages people's attention the most today. This is the case because deep learning allows the automation of tasks such as image classification, speech recognition, object detection, and content description. In short, deep learning allows machines to comprehend data in a more advanced way.

For example, deep learning is used in image classification, speech recognition, object detection, and content analysis. The exponential growth can be attributed to several factors:

Key Drivers of Deep Learning Advancements

1. Algorithmic Improvements

The innovations in the algorithms have drastically increased the usability of the deep learning models.

2. New Machine Learning Approaches

The methodologies have evolved over the years, and that has retained the prediction accuracy of the models as well as the decisions made.

3. Specialized Neural Networks

New formats of the neural networks have been created to enable the performance of certain functions such as document translation or photos categorization to these networks.

4. Abundance of Metadata

The availability of such data is practically a revolution and includes the following types:

- Streaming data from IoT devices;
- Textual data from social media;
- Professional notes like physicians' notes and investigative transcriptions.

These data sources constitute a solid basis for constructing and deep-layered neural networks.

2.1.4 How Deep Learning Works

The deep learning paradigm has the ability to bring revolution in advancement of the field analytics. Deep learning treats the problem differently, as in this approach instead of being explicitly coded the whole program detailing how the problem can be solved, the clear problem, in this case, is to teach the computer to learn how to solve it by itself.

Analytical Methodology In Deep Learning

In traditional analytics, the steps involved in the approach tend to include the following:

1. **Feature Engineering:** Taking out an variable (features) or picking out variables from the data set and recreating ones that can adequately describe the issue.
2. **Model Development:** All the phases of the study until a particular model is completed and picked on.
3. **Parameter Fitting:** Creating the best model that represents the data by changing the parameters of the model.

Although this approach can yield satisfactory predictive systems, it has weaknesses. The performance of the system is highly influenced by the quality of the features and the model. For example, while constructing a model for detecting fraud, one would say:

- You start with parameters that are already set and apply data manipulations in order to generate a model.
- It follows that this process may produce several thousands of variables, which will involve painstaking work to sift through this mass and select the significant ones.
- Inserting new information requires the application of the entire feature engineering exercise all over again.

3. LITERATURE SURVEY

1. Traditional Statistical Models

Early studies relied on **time-series models** and **statistical techniques** to predict electricity demand. These models were effective in capturing seasonality but struggled with non-linear trends.

SARIMA for Electricity Demand Forecasting (Andoh et al., 2021)

- **Method:** Seasonal Auto-Regressive Integrated Moving Average (SARIMA)
- **Concept:** Applied SARIMA for electricity demand forecasting in **Ghana**, considering seasonal variations.
- **Key Contribution:** Improved prediction accuracy by incorporating seasonal effects.
- **Limitations:** Ineffective for short-term forecasting due to assumptions of stationarity.

2. Machine Learning Approaches

Machine learning has significantly improved forecasting accuracy by modeling **non-linear** relationships.

Random Forest for Electricity Demand Prediction (Geetha et al., 2021)

- **Method:** Random Forest (RF)
- **Concept:** Proposed **RF-based supervised learning** for forecasting peak electricity demand using **smart meter data**.
- **Key Contribution:** Improved prediction performance by utilizing an ensemble approach.
- **Limitations:** High computational cost due to multiple decision trees.

3. Deep Learning-Based Approaches

Deep learning techniques, particularly **Long Short-Term Memory (LSTM)** and **Convolutional Neural Networks (CNNs)**, have become dominant in time-series electricity forecasting.

Hybrid CNN-LSTM for Load Forecasting (Alhussein et al., 2020)

- **Method:** CNN + LSTM Hybrid Model
- **Concept:** Combined CNN and LSTM networks for **short-term load forecasting**.
- **Key Contribution:** **Enhanced feature representation** and **sequential learning capabilities**.
- **Limitations:** Requires large datasets for training.

4. Hybrid Approaches

Hybrid models leverage multiple methodologies, integrating statistical, machine learning, and deep learning techniques.

Hybrid ARIMA-ANN for Load Forecasting (Velasco et al., 2019)

- **Method:** ARIMA + Artificial Neural Network (ANN)
- **Concept:** Developed a **hybrid ARIMA-ANN model** for predicting the **next day's electricity load**.
- **Key Contribution:** **Combined interpretability (ARIMA) with prediction power (ANN)**.
- **Limitations:** High complexity in model tuning.

4. SYSTEM ANALYSIS

4.1 EXISTING SYSTEM

The rising global population has led to a surge in electricity consumption, creating greater demand for energy. Power companies must ensure they generate sufficient electricity to meet customer needs, which requires accurate forecasting tools. As residential electricity usage increases, load imbalances often result in power outages. Existing prediction models, however, lack the precision and efficiency needed to manage this growing demand effectively. This highlights the need for a more reliable and advanced predictive system that can better forecast energy requirements. A robust model would not only help prevent outages but also enable energy providers to optimize production, ensuring a stable supply while addressing the growing demands of residential electricity consumption.

Disadvantages of Existing system

1. High costs for new forecasting tools.
2. Complexity in model development and maintenance.
3. Limited accuracy of current prediction models.
4. Risk of data inaccuracies affecting forecasts.
5. Dependence on historical data may skew results.
6. Insufficient real-time monitoring capabilities.
7. Potential for increased operational costs.
8. Vulnerability to external disruptions (e.g., disasters).
9. Resistance from stakeholders to adopt changes.
10. Environmental impacts from increased energy production.

4.2 PROPOSED SYSTEM

So author of this paper employing combination of multiple algorithms which is forecasting electricity consumption with an R2score (accuracy) of over 97%. In propose paper for accurate forecasting author selecting features which are not highly co-related by employing features selection algorithm called 'Maximal Information Coefficient' (MIC). This algorithm will calculate Spearman Rank for each features and then select only those features whose score is not high. High score means features are highly co-related to each other which must be avoided.

In propose paper author combining Convolution Neural Network (CNN), Bidirectional Long Short Term Memory (BILSTM) and Self Attention (SA). CNN will be used to integrate temporal feature extraction from input training features, BILSTM will capture correlated features and then intrinsic feature analysis using SA mechanisms which will enhancing predictive accuracy.

Details of CNN, BILSTM and SA concept you can read from base paper. In propose paper author has trained models for 1 minute, 1 hour and 1 day but it's difficult to implement all this models so we have created propose algorithm to forecast electricity consumption for 1 day.

Propose CNN-BILSTM-SA algorithm performance is evaluated in terms of R2score, RMSE (root mean square error), and MAE (mean absolute error). RMSE and MAE refers to difference between original and predicted values so the lower the difference the better is the algorithm. R2score refers to accuracy for forecasting algorithm whose values between 0 and 1. R2score closer to 1 will be consider as best model. Propose algorithm is compare with many existing algorithms such as Linear Regression, SVM, LSTM and many more.

Advantages of Proposed System

- **Better prediction & adaptability:**

Accurate forecasting with ability to handle changing consumption patterns.

- **Improved reliability:**

Stable grid with fewer outages through better load management.

- **Efficient resource use:**

Optimized power generation, distribution, and real-time monitoring.

- **Cost reduction & profit:**

Lower operational costs with improved profitability.

5. FUNCTIONAL REQUIREMENTS

Functional requirements form an important part in the sphere of software and systems engineering as it determines the conditions which must be fulfilled in order for the system or any of its components to perform their particular tasks. Explain how inputs are transformed into outputs and what the system is expected to achieve once it is completed. Analytic computations, data creation and even some operational processes would be some instances of these requirements. Functional requirements concern the system in its aspects including but not limited to the performance capabilities while non-functional requirements address specific components and capabilities within a system, such as performance, security, or reliability.

Functional requirements are typically put forth as the condition “the system must do ,” whereas for non-functional requirements the statement is put in the form of “the system shall be .” These functional requirements are provided in the system design, while non- functional requirements are maintained in the systems’ architectures. Therefore functional requirements controls the application architecture, and non-functional requirements control the technical architecture.

For the cases above and in real life, functional requirements can be confirmed and further refined through the use cases where the situation is first described. Clients provide their expectations and analysts try to observe, interact and record these expectations. These use cases are instrumental in generating the functional requirements and ensuring that the right functions are performed by the system. These steps of analyses, modeling and validation help to ensure that the requirements are met with expectation of the stakeholders and are possible to be met in reality.

5.1 SYSTEM FUNCTIONAL REQUIREMENTS

1. Data Collection and Input Handling:

- The system should collect and process residential electricity consumption data from smart meters and external sources in real-time or at scheduled intervals.

2. Data Preprocessing:

- The system should normalize the input data using the Min-Max scaling method to ensure consistent data ranges and improve model performance.

3. Feature Selection:

- The system should use the Maximal Information Coefficient (MIC) algorithm to select relevant features and remove highly correlated features to reduce redundancy.

4. Temporal Feature Extraction:

- The Convolutional Neural Network (CNN) should extract temporal features from the input data to capture patterns over time.

5. Sequence Learning:

- The Bidirectional Long Short-Term Memory (BiLSTM) component should process data in both forward and backward directions to capture long-term dependencies in electricity consumption patterns.

6. Attention Mechanism:

- The Self-Attention (SA) module should assign importance weights to different parts of the input sequence to enhance predictive accuracy.

7. Prediction Generation:

- The system should generate short-term (hourly) and long-term (daily) electricity consumption predictions based on the processed data.

8. Performance Evaluation:

- The system should calculate and display key performance metrics such as R^2 score, Root Mean Square Error (RMSE), and Mean Absolute Error (MAE).

9. Visualization:

- The system should provide graphical representations of actual vs predicted consumption data for easy interpretation and analysis.

10. User Interface and Reporting:

- The system should allow users (e.g., administrators and analysts) to access and download prediction results and performance reports through a user-friendly interface.

6. NON-FUNCTIONAL REQUIREMENTS

The non-functional requirements of a system indicate the maintained quality or response time of a system or application. For instance, a user may be configured to a system and once selected the system is expected to respond within a certain time frame: 3 seconds for 10,000 users concurrent. In a brief statement, these types of requirements explain how the system is going to fulfill its operations.

Just like deadlines in real life, they are a must while working on any product or delivering services. Many would refer to these as constraints, as these do set limitations or boundaries within which the design needs to operate. One notion of philosophy in design states ‘out of sight, out of mind’ and these graphic leaders allow exactly that so the consumer focuses on the primary objectives. Without NFRs, consumers could easily become unsatisfied due to the inefficiency of a product.

6.1 REQUIREMENTS

1. Performance:

- The system should generate electricity consumption predictions within **5 seconds** for daily forecasts and **1 second** for hourly forecasts.

2. Scalability:

- The system should be able to handle an increasing volume of data without significant degradation in performance. It should support up to **1 million data points** per day.

3. Reliability:

- The system should maintain at least **99.5% uptime** and handle failures gracefully without data loss or disruption.

4. Accuracy:

- The model should maintain an R^2 score of at least **97%** to ensure high predictive accuracy.

5. Security:

- The system should protect user data through **encryption** and ensure secure access with **role-based authentication**.

6. Maintainability:

- The system code and architecture should be modular and well-documented to facilitate easy maintenance and upgrades.

7. Usability:

- The user interface should be intuitive and easy to navigate, requiring no more than **2 minutes** of training for a new user.

8. Compatibility:

- The system should be compatible with multiple platforms, including **Windows, MacOS, and Linux**, and accessible via web browsers and mobile devices.

9. Availability:

- The system should be available for use **24/7** with automatic recovery from hardware or software failures.

10. Data Storage:

- The system should support long-term storage of at least 5 years of historical data with backup and recovery options available.

7. FEASIBILITY STUDY

The feasibility study succinctly addresses whether the envisaged proposal is workable by presenting an outline and cost figures for the project. This phase is very important in determining whether the system is in concurrence with the objectives of the organization and whether it is appropriate. For purposes of carrying out a feasibility study, there need to be always criteria to be satisfied, in this case, that of understanding the system requirements. It covers the following areas:

1. Economic Feasibility:

This assesses the economic worth of the system. The expenditures incurred in the engagement of such R&D activities should be able to cover the cost incurred. The system is constructed in a way that eliminates many unnecessary costs by using free alternatives and restricting tailor made procurements.

2. Technical Feasibility:

This assesses how the system's technical needs correspond to what is available. The completion of the system development avoids many technical specifications to minimize the alteration of the system at the time of implementation.

3. Social Feasibility:

This measures a system in terms of the aptness or readiness of users to use the system. Training and tutorials build users' self familiarity with the system aiding their seamless adoption of the system in their day to day work processes. User encouragement to give feedback increases the chances of enhanced adoption experience.

8. REQUIREMENT SPECIFICATION

8.1 Hardware Requirements

- PROCESSOR - Intel Core i3/i5
- RAM - 8 GB
- HARD DISK (SSD) - 512 GB

8.2 Software Requirements

- Operating system : Windows 11/ 10
- Coding Language : PYTHON, Jupyter Notebook

9 SYSTEM DESIGN

9.1 METHODOLOGY

Data Collection and Preprocessing

1. **Dataset Source:**

- The dataset, from the UCI repository, includes sub-meter readings for various household devices.
- **Sub-meters:** Three main sub-meters measure usage across devices like water heaters, dishwashers, and washing machines.

2. **Data Cleaning and Normalization:**

- **Date and Time Conversion:** Convert timestamps to numerical values for efficient model processing.
- **Normalization:** Use MinMax scaling to standardize feature ranges, ensuring consistency in model input.

3. **Feature Selection (MIC):**

- Apply the Maximal Information Coefficient (MIC) to rank features based on correlation scores, selecting the top eight features with minimal redundancy.

Model Architecture - CNN-BiLSTM-SA

1. **Convolutional Neural Network (CNN):**

- **Role:** CNN captures local temporal patterns in the consumption data.
- **Application:** Extracts features that highlight time-based changes in electricity usage.

2. **Bidirectional Long Short-Term Memory (BiLSTM):**

- **Role:** BiLSTM identifies forward and backward dependencies, essential for sequence data like electricity consumption.
- **Advantage:** Improves the model's ability to understand trends across both past and future contexts.

3. Self-Attention (SA) Mechanism:

- **Role:** The attention layer weighs each feature's importance, allowing the model to focus on the most predictive factors.
- **Impact:** Enhances accuracy by enabling the model to adaptively prioritize critical features.

Model Training and Evaluation

1. Train-Test Split:

- **Data Allocation:** Use 80% of the dataset for training and 20% for testing to evaluate model generalizability.
- **Model Training:** Train the CNN-BiLSTM-SA model with selected features and evaluate using RMSE, MAE, and R^2 scores.

2. Performance Metrics:

- **R^2 :** Measures the proportion of variance explained by the model; values closer to 1 indicate high predictive accuracy.
- **RMSE and MAE:** Evaluate the error margin between predicted and actual consumption; lower values indicate better model performance.

3. Comparison with Baseline Models:

- **SVM and Linear Regression:** Achieved lower R^2 scores (62% and 45%, respectively), indicating subpar performance.
- **LSTM and CNN-BiLSTM-SA:** Outperformed traditional models, with CNN-BiLSTM-SA achieving the highest accuracy (97%).

Results and Analysis

1. Model Performance Summary:

- CNN-BiLSTM-SA achieved a 97% R^2 score, with low RMSE and MAE values, demonstrating its accuracy in electricity consumption forecasting.

2. Forecasting Visualization:

- Display a line chart comparing actual vs. predicted consumption values, showing that CNN-BiLSTM-SA closely aligns with real data.

3. Metric Comparison Table:

- Summarize R^2 , RMSE, and MAE scores across all models, highlighting CNN-BiLSTM-SA as the top-performing model.

Extension with CNN-BIGRU-SA

1. Gated Recurrent Unit (GRU):

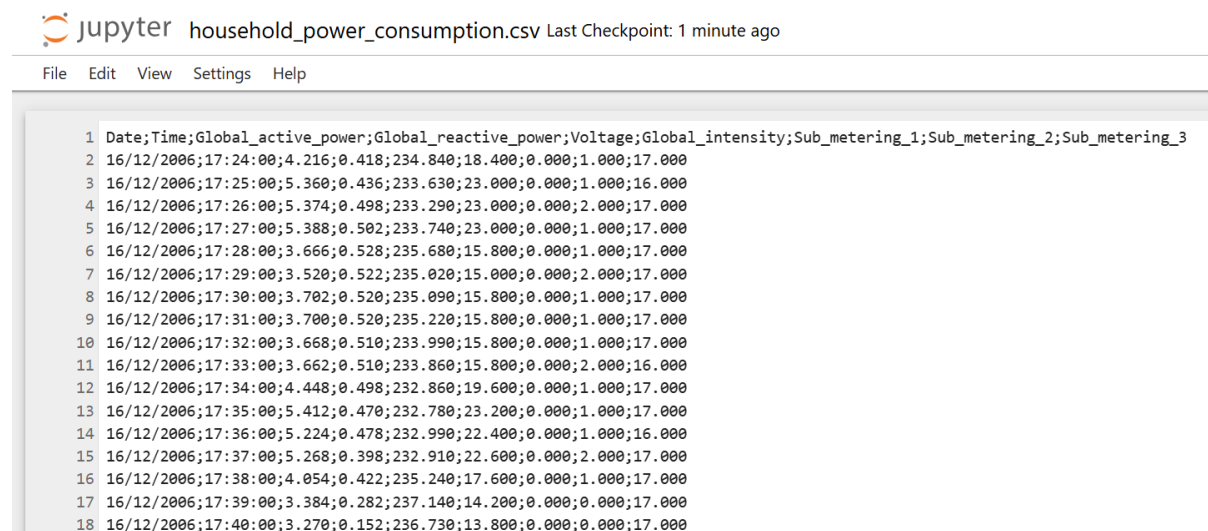
- GRU offers a streamlined architecture compared to BiLSTM, focusing on speed and efficiency.
- **Enhanced Model (CNN-BIGRU-SA):** Achieved slightly higher accuracy with 97.8%, offering faster processing and improved performance.

9.1 DATASET:

To implement this project author has used Electricity Consumption dataset from UCI repository which can be download from below URL

<https://archive.ics.uci.edu/dataset/235/individual+household+electric+power+consumption>

In below screen showing dataset details



```
jupyter household_power_consumption.csv Last Checkpoint: 1 minute ago
File Edit View Settings Help

1 Date;Time;Global_active_power;Global_reactive_power;Voltage;Global_intensity;Sub_metering_1;Sub_metering_2;Sub_metering_3
2 16/12/2006;17:24:00;4.216;0.418;234.840;18.400;0.000;1.000;17.000
3 16/12/2006;17:25:00;5.360;0.436;233.630;23.000;0.000;1.000;16.000
4 16/12/2006;17:26:00;5.374;0.498;233.290;23.000;0.000;2.000;17.000
5 16/12/2006;17:27:00;5.388;0.502;233.740;23.000;0.000;1.000;17.000
6 16/12/2006;17:28:00;3.666;0.528;235.680;15.800;0.000;1.000;17.000
7 16/12/2006;17:29:00;3.520;0.522;235.020;15.000;0.000;2.000;17.000
8 16/12/2006;17:30:00;3.702;0.520;235.090;15.800;0.000;1.000;17.000
9 16/12/2006;17:31:00;3.700;0.520;235.220;15.800;0.000;1.000;17.000
10 16/12/2006;17:32:00;3.668;0.510;233.990;15.800;0.000;1.000;17.000
11 16/12/2006;17:33:00;3.662;0.510;233.860;15.800;0.000;2.000;16.000
12 16/12/2006;17:34:00;4.448;0.498;232.860;19.600;0.000;1.000;17.000
13 16/12/2006;17:35:00;5.412;0.470;232.780;23.200;0.000;1.000;17.000
14 16/12/2006;17:36:00;5.224;0.478;232.990;22.400;0.000;1.000;16.000
15 16/12/2006;17:37:00;5.268;0.398;232.910;22.600;0.000;2.000;17.000
16 16/12/2006;17:38:00;4.054;0.422;235.240;17.600;0.000;1.000;17.000
17 16/12/2006;17:39:00;3.384;0.282;237.140;14.200;0.000;0.000;17.000
18 16/12/2006;17:40:00;3.270;0.152;236.730;13.800;0.000;0.000;17.000
```

In above dataset screen first row contains dataset column names and remaining rows contains dataset values and each row has 3 electricity consumption values such as ‘Sub-meter1, submeter2 and sub-meter3’ where meter1 refers to consumption of ‘Water heater, dish washer’ and meter 2 consumption refers to ‘washing machine and other equipment’s’. You can red paper to know about different equipment’s using different meter consumption. In above dataset

we are summing all 3 sub-meters values as target electricity consumption. So by using above dataset we are evaluating performance of propose and existing algorithms

9.3 SYSTEM ARCHITECTURE:

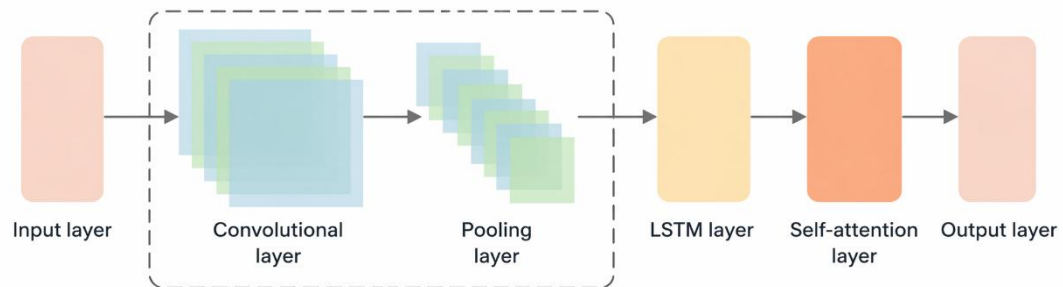


Figure 1 Architecture

10 UML DIAGRAMS

UML is a model while UML is a modeling language. Unified Modeling Language is an internationally accepted modeling palette which makes it easy to visually express plans, implemented ideas and facts in terms of a system- OO. It is a language, so it can be spoken and written. It is a standardized language and there is no national ownership.

UML comprises of two basic aspects: a Meta-model and Notation. This has further enabled the specification, visualization, construction and documentation of software artifacts. It moreover fosters business modeling and other non-software systems, placing emphasis graphical notations for better communication of designs of software systems.

UML adopts the best engineering practices which have been found useful in the modeling of complex and large systems. It is a must in object oriented software engineering for the entire process to be clear, consistent and understandable.

Goals of UML

The primary objectives of UML include:

1. Enhancing the ability to create and communicate meaningful models by providing an appropriate visual modeling language.
2. Introducing provisions for the extendibility and specialization of the basic concepts.
3. Guaranteeing freedom with respect to particular programming languages and specific development methodologies.
4. Providing a helpful underpinning to make clear the language.
5. Facilitating the development of the market of the object-oriented tools.
6. Making it possible to use more sophisticated development concepts such as collaborations, frameworks, patterns and components.

10.1 USE CASE DIAGRAM

A use case diagram in Unified Modeling Language (UML) belongs into class of behavioral diagrams and is obtained through use-case analysis. It graphically represents the functionality of the system by depicting the interactions of the actors towards the achievement of their goals which are modeled as use cases. And, it provides an overview of the dependencies between these use cases.

The main objective of use case diagrams is to indicate what tasks the system performs for each of its actors, so that, the tasks and responsibilities of the actors in the system are better understood.



Figure 2 Use case Diagram

10.2 CLASS DIAGRAM

As it belongs to specific types within the Unified Modeling Language (UML), a class diagram is categorized as a static structure diagram in computer software construction. This is a system modeling method that models a particular system by displaying its classes (or the sets of objects), their features, methods (or functions), and their inter-relationships.

Class diagrams portray the containment and interaction of classes and provide no ambiguity about the system's blueprint. This is why they are a valuable resource for the analysis and design of object-oriented software systems.

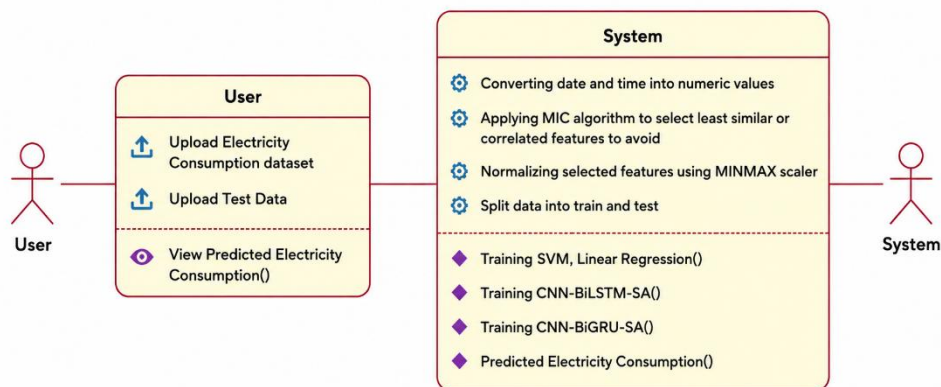


Figure 3 Class Diagram

10.3 SEQUENCE DIAGRAM

UML Sequence diagram belongs to a family of interaction diagrams – dynamic models, and documentation as well – that demonstrate how different processes work, connect and in which order they do so.

Sequence or an event diagram, it is in most cases called an event scenario or timing diagram. It, therefore, provides an elaborated view of the chronological order of events, how objects cooperated in functions over a given period in order to realize a particular process, or in other words, how the objects of the given system behaved during the realized process. These diagrams are great for depicting the behavior of the system and the subtleties of its interactions.

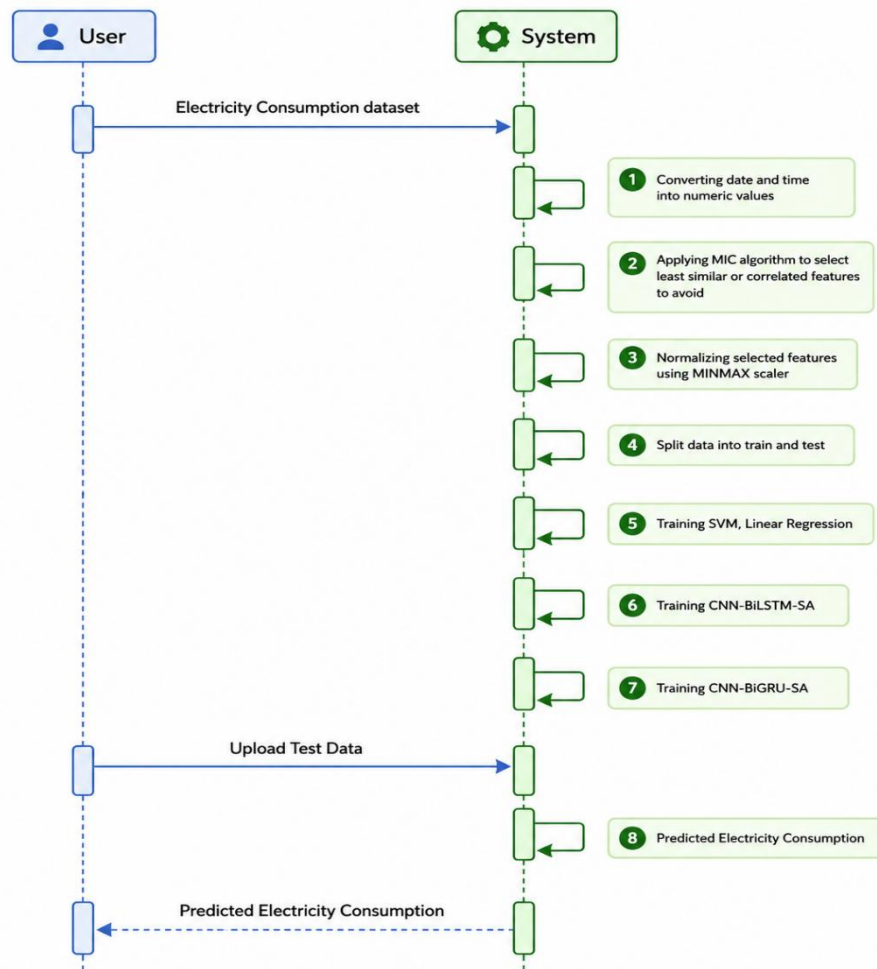


Figure 4 Sequence Diagram

10.4 COLLABORATION DIAGRAM

A collaboration diagram UML is developed to show the links and communication between objects in the system. As opposed to sequence diagrams that chronologically document the sending of messages, collaboration diagrams concentrate on the design and construction of the objects which are the center of the communication at the time of its development.

System objects are endowed with different characteristics, and many of them interrelate with one another and form relationships. The collaboration diagram or the communication diagram depicts these relations graphically in order to give information about the object structure of the system. It assists in appreciating the reason for the existence of some objects and their mode of integration with others in the performance of certain defined functions.

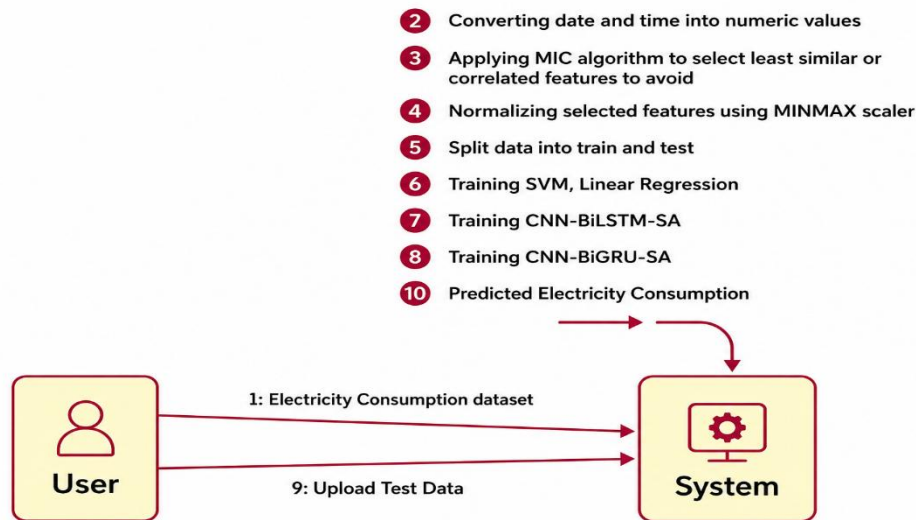


Figure 5 Collaboration Diagram

10.5 ACTIVITY DIAGRAM:

The activity diagram also contains cases of the communication and interactions of objects for the functional purpose. They include a variety of cases such as events and actions in terms of how they flow as well as how they reach the various systems edges or items.

UML activity diagrams outline the business requirements of a process from the user perspective in high level details and also the operational processes of the system explaining how different functional units operate in the context of sequential stages. They are a logically inseparable portion of each process model considering the order of operations and making decisions at each logical point associated with movement of information.

An activity diagram is a fundamental tool in both systems modeling and business processes modeling since it allows any operation to be tracked in a sequential visual representation of its flow.

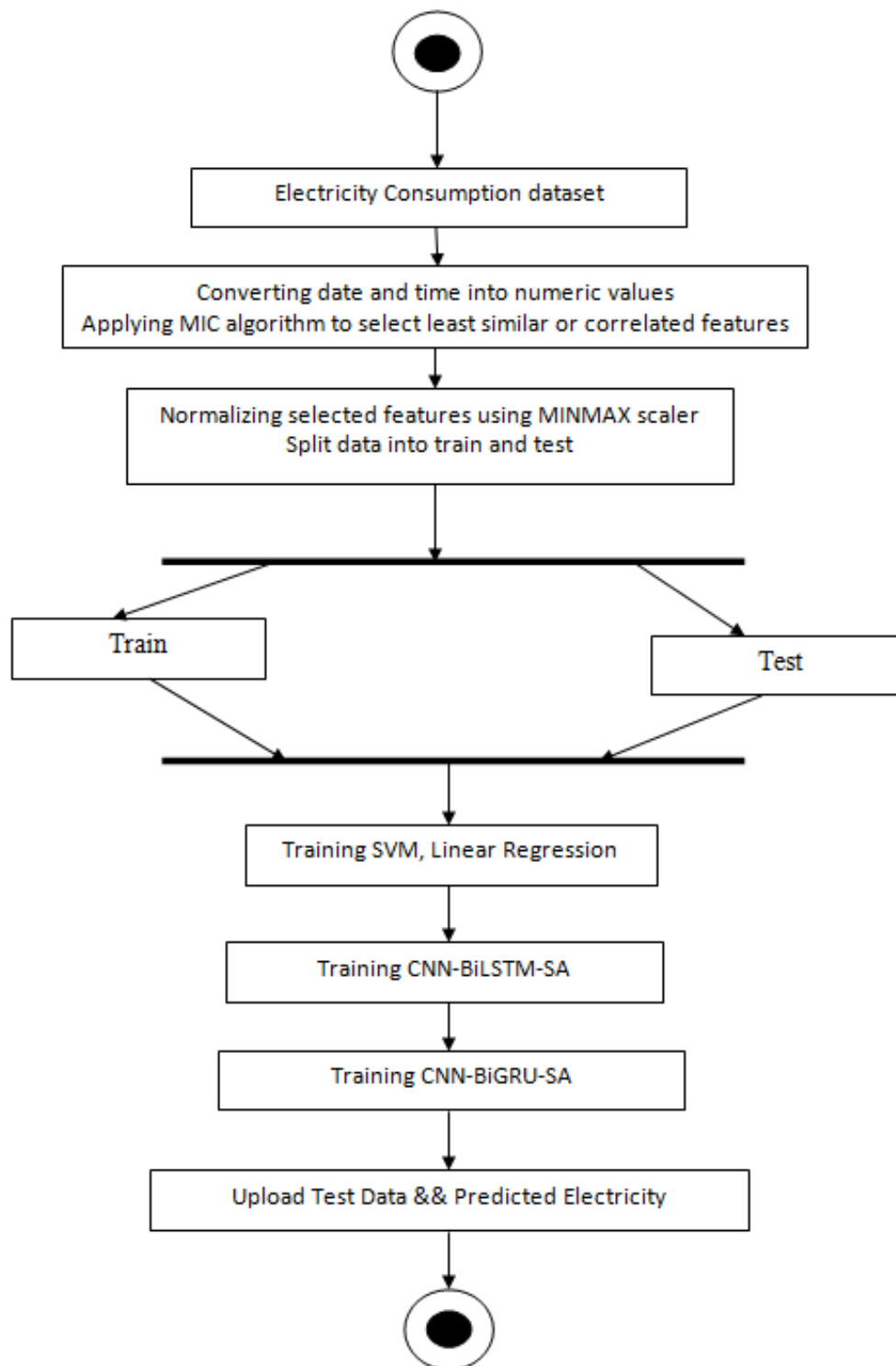


Figure 6 Activity Diagram

11. TECHNOLOGY DESCRIPTIONS

11.1 PYTHON FEATURES

Python Characteristics Easy to learn and easy to use:

Being beginner oriented and developer 100 percent focused, python is relatively easy to learn and use.

Expressive Language:

The coding language of Python is declared as highly expressive as it is claimed that it is easy to read and to write.

Interpreted Language:

The reason for this is because Python makes use of an interpreter which takes in the line of code and executes it although it may bring out a few challenges in the debugging of codes.

Cross-platform Compatibility:

This goes to show that the system has the capacity of being used in different kinds of operating systems such as Windows, Linux, Unix and mac operating systems.

Free and Open Source:

Any person who wishes to avail Python can find it free for downloading and It is easy to acquire and modify the source code.

11.2 PYTHON APPLICATIONS

Since Python is a multipurpose language, it can be used in almost all spheres of software development. Other notable fields where Python has its application include:

1. web Applications:

With its libraries for internet protocols as HTML, XML, JSON, and many others, and frameworks such as Django, Flask, Pyramid, Python is among the most popular languages for web development. Such include PythonWikiEngines, Pycoco and others.

2. Desktop GUI Applications:

Desktops, GUI can be developed using Tkinter, wxWidgets, Kivy and PyQt. For multi touch applications across platforms Kivy is good for such purpose.

3. Software Development:

Programming in Python is widely used in software development as an auxiliary language for such activities as build control, testing, management, etc.

4. Scientific and Numeric Computing:

Libraries like **SciPy**, **Pandas**, and **IPython** are popular for scientific computing, engineering, and mathematical analysis.

11.4 WHY PYTHON

Python is a versatile, user-friendly, and powerful programming language widely used across industries. Here's why Python is so popular:

- **Cross-Platform Compatibility:** Python runs on various platforms, including Windows, Mac, Linux, and Raspberry Pi, making it highly portable.
- **Simple and Readable Syntax:** Python's syntax is simple, clean, and similar to the English language, making it easier for developers to learn and write code.
- **Less Code, More Functionality:** Python allows developers to accomplish tasks with fewer lines of code compared to other programming languages.
- **Interpreter-Based Language:** Python runs on an interpreter, meaning that code can be executed immediately after being written. This makes prototyping faster and more efficient.
- **Multiple Programming Paradigms:** Python supports procedural, object-oriented, and functional programming, offering flexibility to developers.

12. SAMPLE CODE

#importing all required python libraries

```
import pandas as pd
```

```
import numpy as np
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.preprocessing import MinMaxScaler
```

```
from sklearn.metrics import mean_squared_error
```

```
from sklearn.metrics import r2_score
```

```
from math import sqrt
```

```
import os
```

```
from pathlib import Path
```

```
from sklearn.metrics import mean_absolute_error
```

```
from sklearn import svm
```

```
from minepy import MINE #loading class to select features using MIC (Maximal Information Coefficient)
```

```
from keras.layers import MaxPooling2D
```

```
from keras.layers import Dense, Dropout, Activation, Flatten, RepeatVector, Bidirectional, LSTM, GRU, AveragePooling2D
```

```
from keras.layers import Convolution2D
```

```
from keras.models import Sequential, Model
```

```
from keras.callbacks import ModelCheckpoint
```

```
import pickle
```

```
import matplotlib
```

```
matplotlib.use('module://matplotlib_inline.backend_inline')
```

```
import matplotlib.pyplot as plt #use to visualize dataset vallues
```

```
from keras.layers import *
```

```
from keras.models import *
```

```

from keras import backend as K

from sklearn.linear_model import LinearRegression

#defining self attention layer

import tensorflow as tf

class attention(Layer):

    def __init__(self, return_sequences=True, name=None, **kwargs):

        super(attention, self).__init__(name=name, **kwargs)

        self.return_sequences = return_sequences


    def build(self, input_shape):

        self.W = self.add_weight(name="att_weight", shape=(input_shape[-1], 1),
initializer="normal")

        self.b = self.add_weight(name="att_bias", shape=(input_shape[1], 1),
initializer="zeros")

        super(attention, self).build(input_shape)


    def call(self, x):

        e = tf.math.tanh(tf.matmul(x, self.W) + self.b)

        a = tf.nn.softmax(e, axis=1)

        output = x * a

        if self.return_sequences:

            return output

        return tf.reduce_sum(output, axis=1)


    def compute_output_shape(self, input_shape):

        if self.return_sequences:

            return input_shape

        return (input_shape[0], input_shape[-1])

```

#visualizing graph of electricity consumption of different sub-meters where each sub-meter has consumption for different

#equipments such as water heater, dish washer, washing maching etc

```
plt.figure(figsize=(6,4))

plt.plot(dataset['Sub_metering_1'], label='Sub-Meter 1 Consumption')
plt.plot(dataset['Sub_metering_2'], label='Sub-Meter 2 Consumption')
plt.plot(dataset['Sub_metering_3'], label='Sub-Meter 3 Consumption')
plt.title("Electricity Consumption Recorded by Different Sub-Meters")
plt.xlabel("Number of Records")
plt.ylabel("Electricity Consumption")
plt.legend()
plt.show()
```

#applying dataset processing such as converting date and time into numeric values and then summing all 3

#submeters consumption as single target value to forecast future electricity

```
dataset['Date'] = pd.to_datetime(dataset['Date'], format='%d/%m/%Y', dayfirst=True)
dataset['year'] = dataset['Date'].dt.year
dataset['month'] = dataset['Date'].dt.month
dataset['day'] = dataset['Date'].dt.day
dataset['Time'] = pd.to_datetime(dataset['Time'], format='%H:%M:%S')
dataset['hour'] = dataset['Time'].dt.hour
dataset['minute'] = dataset['Time'].dt.minute
dataset['second'] = dataset['Time'].dt.second

dataset['label'] = dataset['Sub_metering_1'] + dataset['Sub_metering_2'] +
dataset['Sub_metering_3']

dataset.drop(['Date', 'Time', 'Sub_metering_1', 'Sub_metering_2', 'Sub_metering_3'], axis =
1,inplace=True)

dataset.fillna(0, inplace = True)
```

```
dataset
```

```
#applying MIC (Maximal Information Coefficient) algorithm to select least similar or correlated features to avoid
```

```
#poor prediction values
```

```
Y = dataset['label'].to_numpy().ravel() #getting target column
```

```
dataset.drop(['label'], axis = 1,inplace=True)
```

```
columns = dataset.columns
```

```
X = dataset.values #get dataset features
```

```
print("Total features exists in Dataset before applying MIC features Selection algorithm :  
"+str(X.shape[1]))
```

```
mic_scores = []
```

```
mine = MINE()
```

```
for i in range(0, len(columns)-1):#loop and compute mic score for each features
```

```
    mine.compute_score(X[:,i], Y)
```

```
    mic_scores.append((columns[i], mine.mic()))
```

```
# Sort features by MIC score
```

```
mic_scores.sort(key=lambda x: x[1], reverse=True)
```

```
# Select top features
```

```
top_features = [feature for feature, _ in mic_scores[:8]] # Select top 2 features
```

```
X = dataset[top_features]
```

```
print("Total features exists in Dataset before applying MIC features Selection algorithm :  
"+str(X.shape[1]))
```

```
X = dataset.values
```

```
#normalizing selected features using MINMAX scaler
```

```
Y = Y.reshape(-1, 1)
```

```
scaler = MinMaxScaler((0,1))
```

```
scaler1 = MinMaxScaler((0,1))
```

```
X = dataset.values
```

```

X = scaler.fit_transform(X)
Y = scaler1.fit_transform(Y)
ARTIFACTS_DIR.mkdir(exist_ok=True)
with open(ARTIFACTS_DIR / "feature_scaler.pkl", 'wb') as f:
    pickle.dump(scaler, f)
with open(ARTIFACTS_DIR / "label_scaler.pkl", 'wb') as f:
    pickle.dump(scaler1, f)

print(f'Saved scaler artifacts: {ARTIFACTS_DIR / "feature_scaler.pkl"} and
{ARTIFACTS_DIR / "label_scaler.pkl"}')

print("Normalized Features = "+str(X))

#split data into train and test

X_train, X_test, y_train, y_test = train_test_split(X, Y, test_size=0.2)

print("Train & Test Dataset Split")

print("80% records used to train algorithms : "+str(X_train.shape[0]))

print("20% records features used to test algorithms : "+str(X_test.shape[0]))

#training propose CNN-BiLSTM-SA algorithm on training features and then evaluate
performance on 20% test features

#this algorithm is a combination of CNN, BI-LSTM and SA (self attention)

propose_model = Sequential()

propose_model.add(Input(shape=(X_train1.shape[1], X_train1.shape[2], X_train1.shape[3])))

propose_model.add(Convolution2D(32, (1, 1), activation='relu'))

propose_model.add(MaxPooling2D(pool_size=(1, 1)))

propose_model.add(Convolution2D(32, (1, 1), activation='relu'))

propose_model.add(MaxPooling2D(pool_size=(1, 1)))

propose_model.add(Flatten())

propose_model.add(RepeatVector(3))

propose_model.add(attention(return_sequences=True, name='attention'))

propose_model.add(Bidirectional(LSTM(64, activation='relu')))

```

```

propose_model.add(RepeatVector(3))

propose_model.add(Bidirectional(LSTM(64, activation='relu'))))

propose_model.add(Dense(units=256, activation='relu'))

propose_model.add(Dropout(0.3))

propose_model.add(Dense(units=1))

propose_model.compile(optimizer='adam', loss='mean_squared_error')

#function to calculate MSE, RMSE and R2Square from predicted and true values

def calculateMetrics(algorithm, predict, test_labels):

    mse_error = mean_squared_error(test_labels, predict)

    mae_error = mean_absolute_error(test_labels, predict)

    r2_scores = r2_score(test_labels, predict)

    rmse_error = sqrt(mse_error)

    print(algorithm + " MAE : " + str(mae_error))

    print(algorithm + " RMSE : " + str(rmse_error))

    print(algorithm + " R2 : " + str(r2_scores))

#visualizing RMSE performance of all algorithms

height = rsquare

bars = ['SVM', 'Linear Regression', 'Propose CNN-BiLSTM-SA', 'Extension CNN-BiGRU-SA']

y_pos = np.arange(len(bars))

plt.figure(figsize = (9, 3))

plt.bar(y_pos, height)

plt.xticks(y_pos, bars)

plt.xlabel("Algorithms")

plt.ylabel("R2 Score")

plt.title("All Algorithms R2Score Comparison Graph")

plt.tight_layout()

```

```

plt.show()

#display all algorithm performnace

algorithms = ['Existing SVM', 'Existing Linear Regression', 'Propose CNN-BiLSTM-SA',
'Extension CNN-BiGRU-SA']

data = []

for i in range(len(rmse)):

    data.append([algorithms[i], rsquare[i], rmse[i], mae[i]])

data = pd.DataFrame(data, columns=['Algorithm Name', 'R2Score', 'RMSE', 'MAE'])

data

#model_factory.py

class Attention(Layer):

    def build(self, input_shape):

        self.W = self.add_weight(

            name="att_weight",

            shape=(input_shape[-1], 1),

            initializer="normal",

        )

        self.b = self.add_weight(

            name="att_bias",

            shape=(input_shape[1], 1),

            initializer="zeros",

        )

    def call(self, x):

        e = tf.math.tanh(tf.matmul(x, self.W) + self.b)

        a = tf.nn.softmax(e, axis=1)

        output = x * a

        return output

```

```

def build_forecasting_model(recurrent="gru", input_shape=(10, 1, 1)):
    model = Sequential()
    model.add(Input(shape=input_shape))
    model.add(Convolution2D(32, (1, 1), activation="relu"))
    model.add(MaxPooling2D(pool_size=(1, 1)))
    model.add(Convolution2D(32, (1, 1), activation="relu"))
    model.add(MaxPooling2D(pool_size=(1, 1)))
    model.add(Flatten())
    model.add(RepeatVector(3))
    model.add(Attention(return_sequences=True, name="attention"))
    model.add(Bidirectional(LSTM(64, activation="relu")))
    model.add(Dense(units=256, activation="relu"))
    model.add(Dropout(0.3))
    model.add(Dense(units=1))
    model.compile(optimizer="adam", loss="mean_squared_error")
    return model

```

#preprocessing.py

```

dates = pd.to_datetime(working["Date"], format="%d/%m/%Y", dayfirst=True)
times = pd.to_datetime(working["Time"], format="%H:%M:%S")

working["year"] = dates.dt.year
working["month"] = dates.dt.month
working["day"] = dates.dt.day
working["hour"] = times.dt.hour
working["minute"] = times.dt.minute
working["second"] = times.dt.second

```

```

features = working[FEATURE_COLUMNS].astype(float)

#prediction_service.py

scaled = feature_scaler.transform(features.to_numpy())

model_input = np.reshape(scaled, (scaled.shape[0], scaled.shape[1], 1, 1))

scaled_predictions = model.predict(model_input, verbose=0).reshape(-1, 1)

predictions = np.abs(label_scaler.inverse_transform(scaled_predictions).reshape(-1))

#evaluation_service.py

return {

    "MAE": float(mean_absolute_error(actual_values, predicted_values)),

    "RMSE": float(np.sqrt(mean_squared_error(actual_values, predicted_values))),

    "R2 Score": float(r2_score(actual_values, predicted_values)),

    "MAPE": mape,

}

# routes.py

@main.route("/PredictAction", methods=["POST"])

def predict_action():

    selected_model = request.form.get("model_key", DEFAULT_MODEL_KEY)

    uploaded_file = request.files.get("t1")

    result, status_code = build_prediction_response(

        file_storage=uploaded_file,

        model_key=selected_model,

    )

    return render_predict_page(result=result, selected_model=selected_model)

```

13. TESTING

13.1 INTRODUCTION

Software engineers draw a distinction between software fault and software failure. A fault will mean an error that is either a bug in programming or a defect in a certain program's semantic. A fault may or may not result in a failure. A failure on the other hand is such that a software has not performed as expected by the end user. Under certain conditions such as when the faulty code is run on the CPU, or even when it is migrated to another hardware machine or when it is enhanced with new capabilities, faults become failures.

Testing a software application is always essential in the process of analyzing the software with the aim of validating its quality and providing the stakeholders with relevant information. It states the errors that it contains and so guarantees that a software has a certain amount of standards that it fulfills with users and system requirements.

13.1.1 System Testing and Implementation

System testing is concerned with testing the logic of individual module codes and debugging them before their assembly into larger subsystems. This involves:

1. **Module Integration Testing:** This tests the linkages of modules to one another.
2. **System Testing:** This is done to check that the whole system conforms to the requirements provided.
3. **Acceptance Testing:** This goal is to represent the abilities of the system to the customers.

There are two methods that could be used in selecting test cases which include:

- **Black Box Testing** - Testing of the external behavior with respect to the system requirements specification.
- **White Box Testing** - Concentrates on the assessment of internal logic and aims at coverage of the code.

13.1.2 Testing Techniques

Testing discovers mistakes made in the software by executing the software under conditions that were planned in advance. Two main approaches are:

1. Black Box Testing:

Emphasizes functional specification and assesses external behavior without any interference on internal logic. It detects errors such as interface error, the database access error, performance as well.

2. White Box Testing:

Deals with the internal logic by scrutinizing module flow graphs and analyzing logical paths, loops and data structures to be tested

13.1.3 Testing Strategies

To ensure their applications are free from errors and comply with the required functionality, the effective performance of software testing is paramount. Given below are the top primary kinds of software testing strategies used in the software development process:

Unit Testing

Unit testing tends to assure the correctness of the implementation of individual units or components of the software. It is usually conducted to ensure the correctness of internal logic, and that for a particular input element expected output is given. The unit testing is said to be structural and invasive since it depends on the knowledge of the structure of the software developed.

The main objective of developers when using unit testing is to ensure the validity of each decision branch and each internal code path. This guarantees all business logic and configurations function as intended. Unit testing is usually done at the level of components and it helps in detecting faults very early in the development cycle.

A good example would be a system with three modules: a Reputation Module, Route Discovery Module and Audit Module. All of these systems would be put through a separate test. Errors are recognized during this stage, patches are made and the patched executable units are checked before they are incorporated.

Integration Testing

Integration testing is usually done after unit testing and it is concerned with putting several units or components together into one cohesive system as they were meant to work. It is event driven, concentrating on functional interconnections and ultimately the interaction of the components of the system.

While unit testing touches on the performance of each individual component, unit testing is aimed at confirming that all the component systems do not create problems when put together. These tests determine whether there are discrepancies in attached data and also the communication lines between modules so as to enhance unity and harmony in system execution.

System Testing

As the name states, system testing tests the final, integrated software product to ensure everything is functioning according to the requirements. It deals with the global layout and just like the end to end testing, guarantees normal results given the particular components are used in a realistic way.

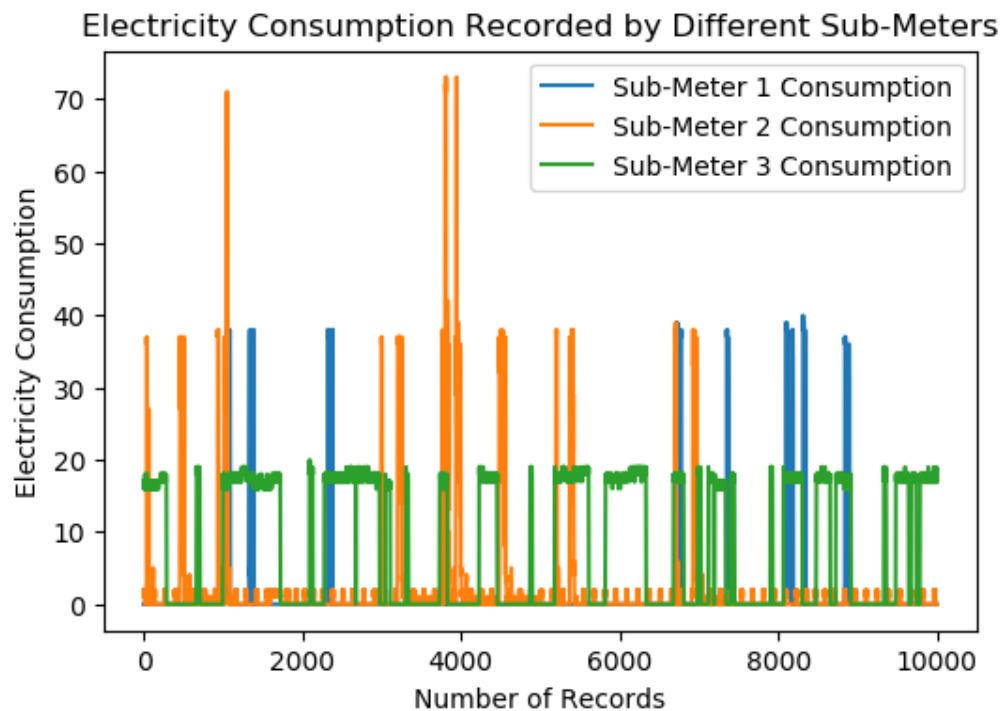
One example of system testing is configuration-oriented system integration testing, which checks how processes interact at various integration points. System testing focuses on process flows and assesses whether the system is reliable and meets the required expectations.

Functional Testing

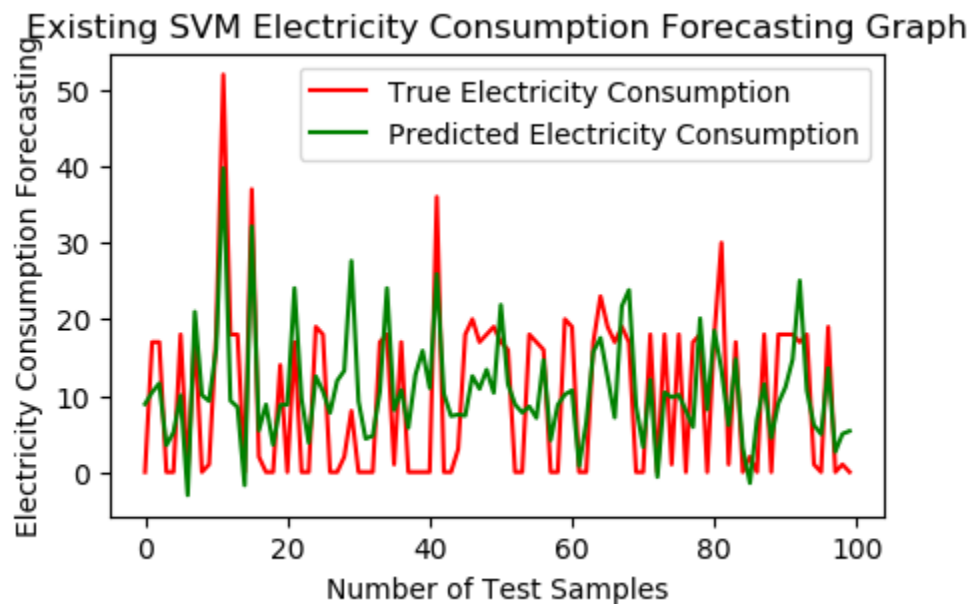
Functional testing establishes whether the system implemented meets the business and technical requirements stated in system documents and users' manuals. It offers a systematic approach for verifying that all required functions have been implemented correctly and as expected by users.

Functional testing was done by using each business process or feature and user interaction against documented requirements. This helps ensure that all features work as they are intended to work without any problems.

14. SCREENSHOTS

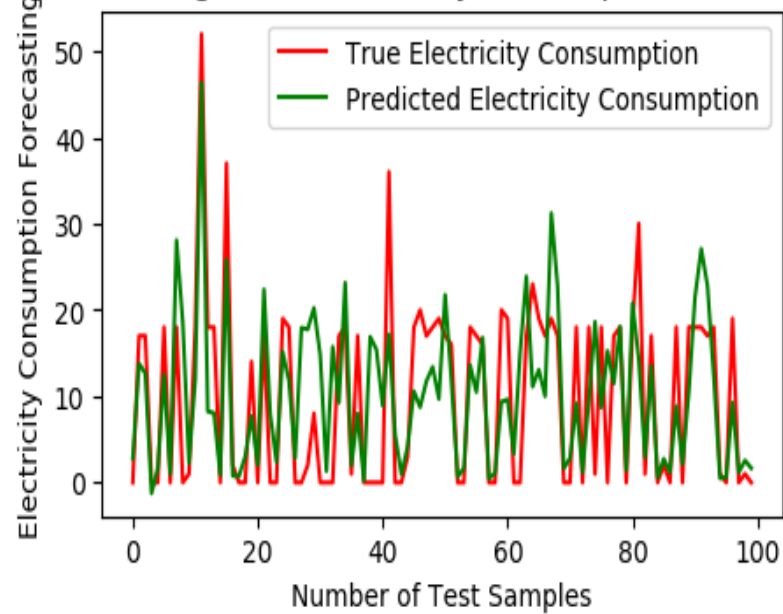


Screen 1 Electricity Consumption



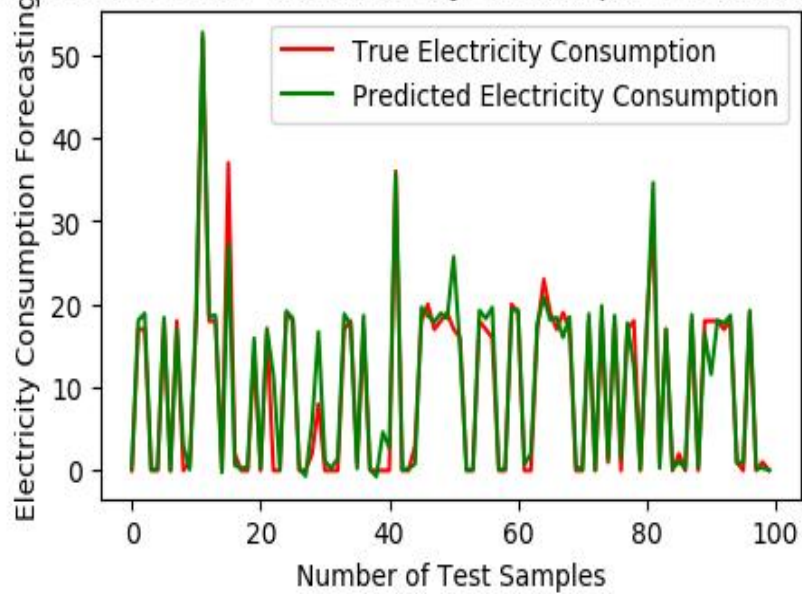
Screen 2 SVM algorithm

Existing Linear Regression Electricity Consumption Forecasting Graph



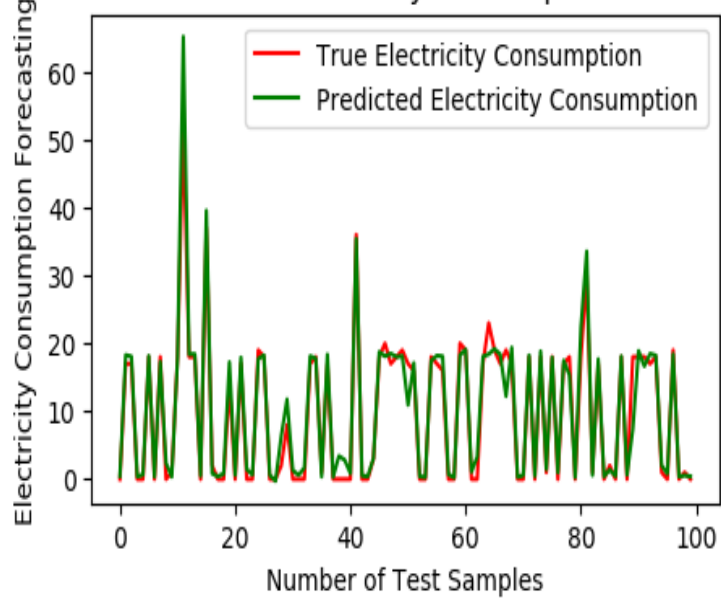
Screen 3 Linear Regression algorithm

Propose CNN-BiLSTM-SA Electricity Consumption Forecasting Graph

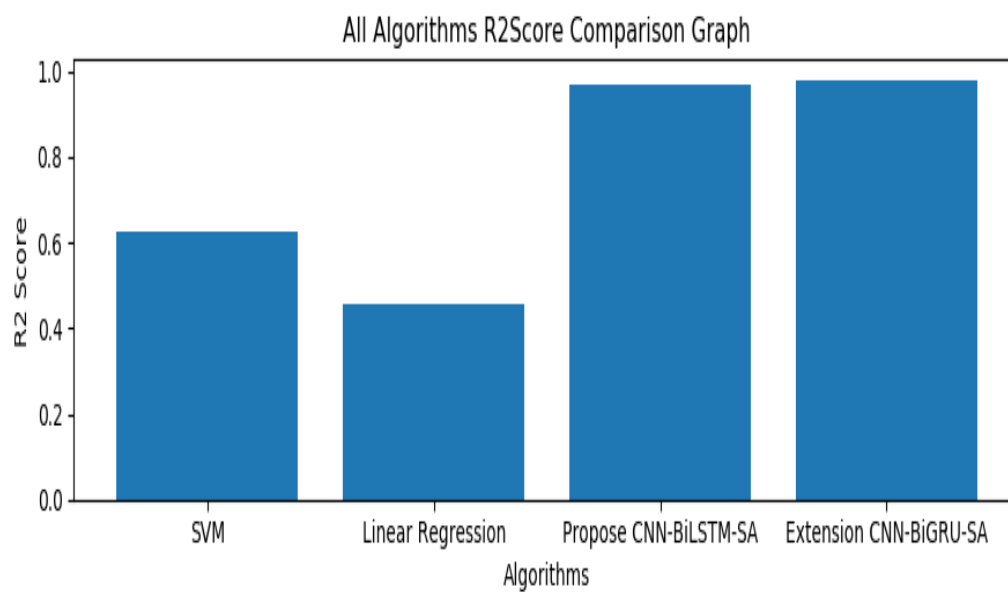


Screen 4 CNN-BiLSTM-SA algorithm

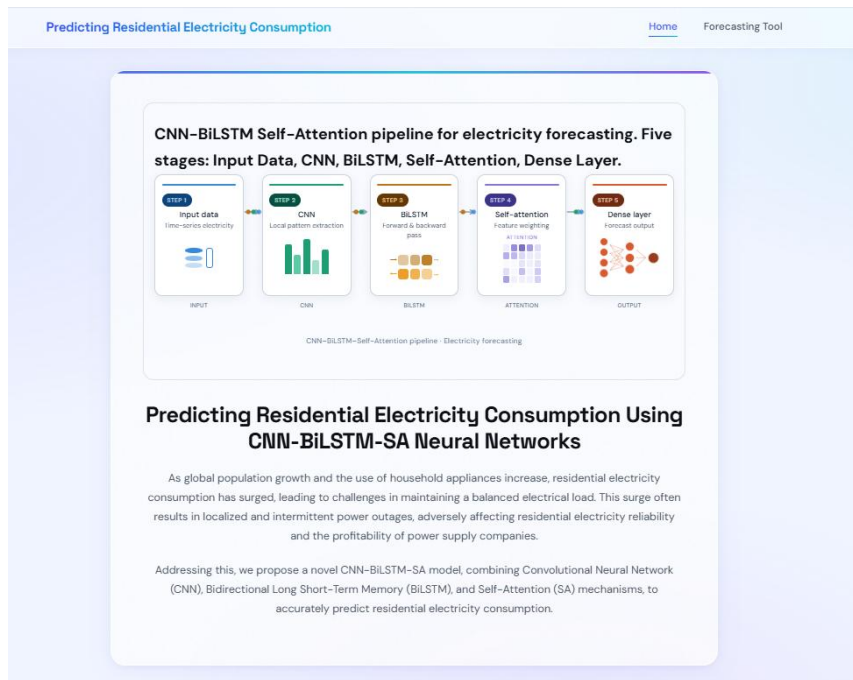
Extension CNN-BiGRU-SA Electricity Consumption Forecasting Graph



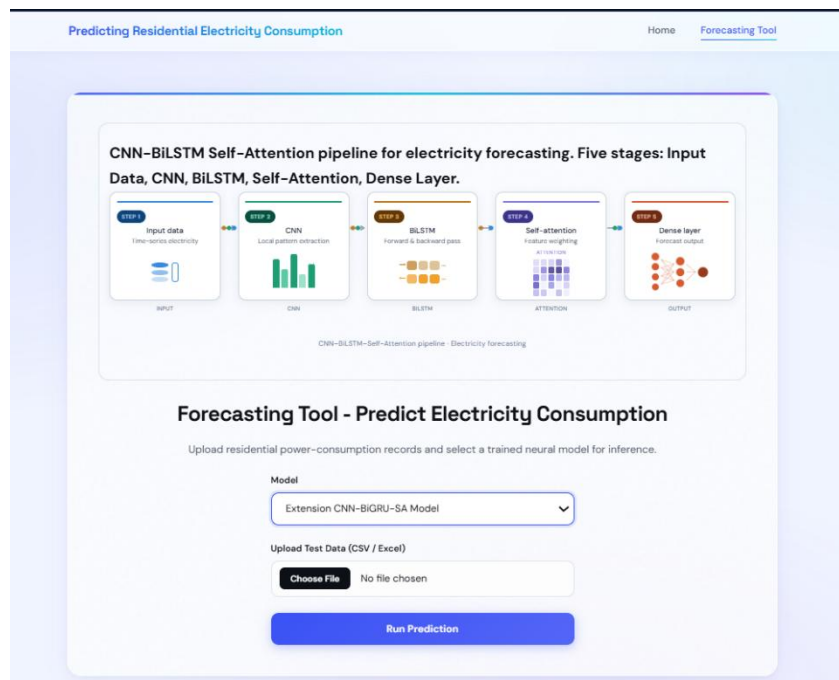
Screen 5 CNN-BiGRU-SA algorithm



Screen 6 RMSE Performance of all algorithms



Screen 7 Home Page



Screen 8 Forecasting Page

CNN-BiLSTM Self-Attention pipeline for electricity forecasting. Five stages: Input Data, CNN, BiLSTM, Self-Attention, Dense Layer.



Forecasting Tool - Predict Electricity Consumption

Upload residential power-consumption records and select a trained neural model for inference.

Model

Extension CNN-BiGRU-SA Model

Upload Test Data (CSV / Excel)

Choose File

No file chosen

Run Prediction

Prediction Dashboard

Extension CNN-BiGRU-SA Model | testData.csv

Download CSV

Download Report

Total Rows

9

Min Prediction

1.8947

Max Prediction

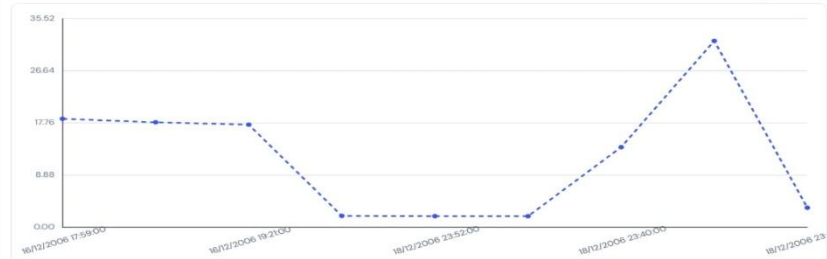
31.7149

Average Prediction

12.0244

Actual vs Predicted Consumption

Predicted



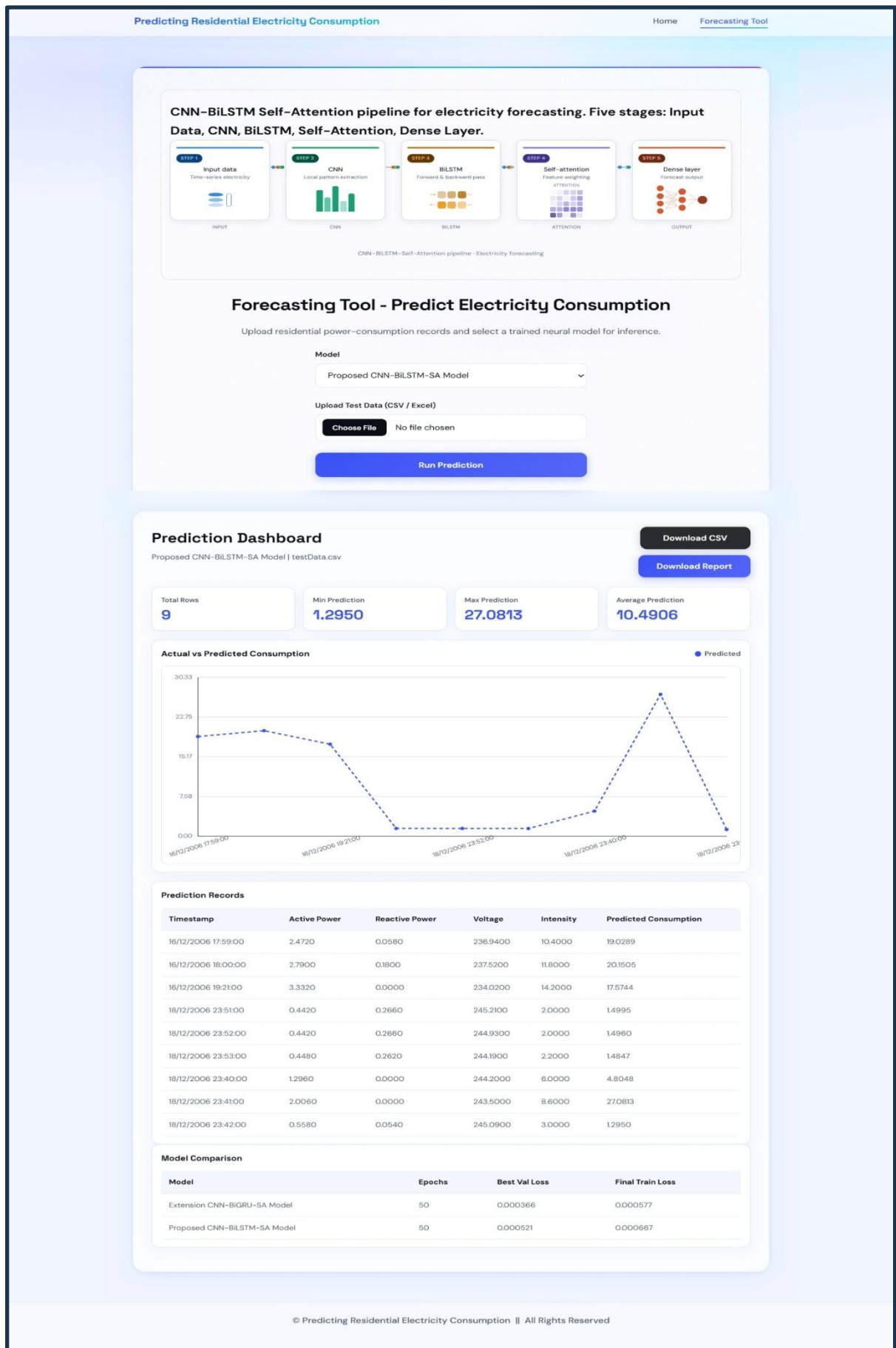
Prediction Records

Timestamp	Active Power	Reactive Power	Voltage	Intensity	Predicted Consumption
16/12/2006 17:59:00	2.4720	0.0580	236.9400	10.4000	18.4596
16/12/2006 18:00:00	2.7900	0.1800	237.5200	11.8000	17.8724
16/12/2006 19:21:00	3.3320	0.0000	234.0200	14.2000	17.4654
18/12/2006 23:51:00	0.4420	0.2660	245.2100	2.0000	19.468
18/12/2006 23:52:00	0.4420	0.2660	244.9300	2.0000	19.014
18/12/2006 23:53:00	0.4480	0.2620	244.1900	2.2000	1.8947
18/12/2006 23:40:00	1.2960	0.0000	244.2000	6.0000	13.6352
18/12/2006 23:41:00	2.0060	0.0000	243.5000	8.6000	31.7149
18/12/2006 23:42:00	0.5580	0.0540	245.0900	3.0000	3.3292

Model Comparison

Model	Epochs	Best Val Loss	Final Train Loss
Extension CNN-BiGRU-SA Model	50	0.000366	0.000577
Proposed CNN-BiLSTM-SA Model	50	0.000521	0.000667

Screen 9 Extension CNN-BiGRU-SA Model Output



15. CONCLUSION

The proposed CNN-BiLSTM-SA algorithm effectively predicts residential electricity consumption with a high R^2 score of 97%, outperforming traditional methods such as SVM and Linear Regression. The algorithm integrates CNN for feature extraction, BiLSTM for handling sequence data, and Self-Attention for improving accuracy by focusing on important features. The Maximal Information Coefficient (MIC) method enhances feature selection, leading to improved predictive performance. The extension model using CNN-BiGRU-SA further improves accuracy, achieving an R^2 score of 97.81%, with lower RMSE and MAE values. The study demonstrates that combining deep learning models with advanced feature selection can significantly enhance electricity consumption forecasting, supporting more efficient energy management and planning.

16. GLOSSARY

Artificial Intelligence (AI): A field of computer science focused on creating systems that can perform tasks requiring human intelligence such as decision-making, problem-solving, and pattern recognition.

Artificial Neural Network (ANN): A computational model inspired by the human brain, consisting of interconnected nodes (neurons) used to recognize patterns and relationships in data.

Auto-Regressive Moving Average (ARMA): A statistical model used for time-series forecasting that combines autoregression and moving average components.

Auto-Regressive Integrated Moving Average (ARIMA): An extension of ARMA that includes differencing to make non-stationary time-series data stationary.

Bidirectional Gated Recurrent Unit (BiGRU): A variant of GRU that processes sequence data in both forward and backward directions for better context understanding.

Bidirectional Long Short-Term Memory (BiLSTM): An advanced LSTM model that processes data in both directions, capturing past and future dependencies in sequences.

Central Processing Unit (CPU): The primary component of a computer that performs most of the processing tasks and executes instructions.

Computer-Aided Design (CAD): Software used for designing and modeling objects in engineering, architecture, and manufacturing.

Convolutional Neural Network (CNN): A deep learning model primarily used for image and spatial data analysis through convolutional layers.

CNN-BiLSTM-SA (Hybrid Model): A combined deep learning architecture that integrates CNN for feature extraction, BiLSTM for sequence learning, and Self-Attention for focusing on important features.

Deep Learning (DL): A subset of machine learning that uses multi-layer neural networks to model complex patterns in large datasets.

Enterprise Resource Planning (ERP): Software systems that integrate and manage core business processes such as finance, supply chain, and human resources.

Gated Recurrent Unit (GRU): A type of recurrent neural network that uses gating mechanisms to efficiently capture dependencies in sequential data.

Graphical User Interface (GUI): A user interface that allows interaction with electronic devices through graphical elements like windows, icons, and buttons.

Integrated Development Environment (IDE): Software that provides tools such as code editors, debuggers, and compilers for software development.

Internet of Things (IoT): A network of interconnected physical devices that collect and exchange data over the internet.

Linear Regression (LR): A basic statistical method used to model the relationship between a dependent variable and one or more independent variables.

Long Short-Term Memory (LSTM): A type of recurrent neural network designed to capture long-term dependencies in sequential data using memory cells.

Machine Learning (ML): A subset of artificial intelligence that enables systems to learn from data and improve performance without explicit programming.

Maximal Information Coefficient (MIC): A statistical measure used to identify both linear and non-linear relationships between variables.

Mean Absolute Error (MAE): A metric that calculates the average absolute difference between predicted and actual values.

MinMaxScaler (Min-Max Normalization Technique): A preprocessing method that scales data into a fixed range, typically between 0 and 1.

Natural Language Processing (NLP): A branch of AI that focuses on enabling computers to understand and process human language.

Operating System (OS): System software that manages hardware resources and provides services for computer programs.

Random Forest (RF): An ensemble learning method that builds multiple decision trees and combines their outputs to improve accuracy.

Root Mean Square Error (RMSE): A performance metric that measures the square root of the average squared differences between predicted and actual values.

Scikit-learn (sklearn): A popular Python library used for implementing machine learning algorithms and data preprocessing techniques.

Seasonal Auto-Regressive Integrated Moving Average (SARIMA): An extension of ARIMA that accounts for seasonal patterns in time-series data.

Self-Attention (SA): A mechanism that allows models to focus on important parts of input data by assigning different weights to different elements.

Support Vector Machine (SVM): A supervised learning algorithm used for classification and regression tasks by finding an optimal separating boundary.

Unified Modeling Language (UML): A standardized modeling language used to visualize and design software systems.

University of California Irvine (UCI Repository): A well-known repository that provides datasets for machine learning research and experimentation

16. BIBLIOGRAPHY

- [1] P. Duffy, C. Fitzpatrick, T. Conway, and R. P. Lynch, “Energy sources and supply grids—The growing need for storage,” in *Energy Storage Options and Their Environmental Impact (Issues in Environmental Science and Technology)*, vol. 46, Cambridge, U.K.: R. E. Hester and R. M. Harrison, 2018, pp. 1–41, doi: 10.1039/9781788015530-00001.
- [2] C. C. Ding, “The dynamic effect and regional difference of the industrialization and urbanization on the energy consumption intensity in China,” *J. Hebei Univ. Econ. Bus.*, vol. 36, pp. 47–54, Mar. 2015.
- [3] B. Hojjati, “Global energy consumption driven by more electricity in residential, commercial buildings,” U.S. Energy Inf. Admin., Washington, DC, USA, Tech. Rep. 38, 2019.
- [4] S. S. Pappas, L. Ekonomou, P. Karampelas, D. C. Karamousantas, S. K. Katsikas, G. E. Chatzarakis, and P. D. Skafidas, “Electricity demand load forecasting of the hellenic power system using an ARMA model,” *Electr. Power Syst. Res.*, vol. 80, no. 3, pp. 256–264, Mar. 2010.